

Lab Lecture



REST API with Python

Dr. Ei Ei Thu

Associate Professor

Faculty of Computer Science

Outlines



Introduction



Python Flask Example



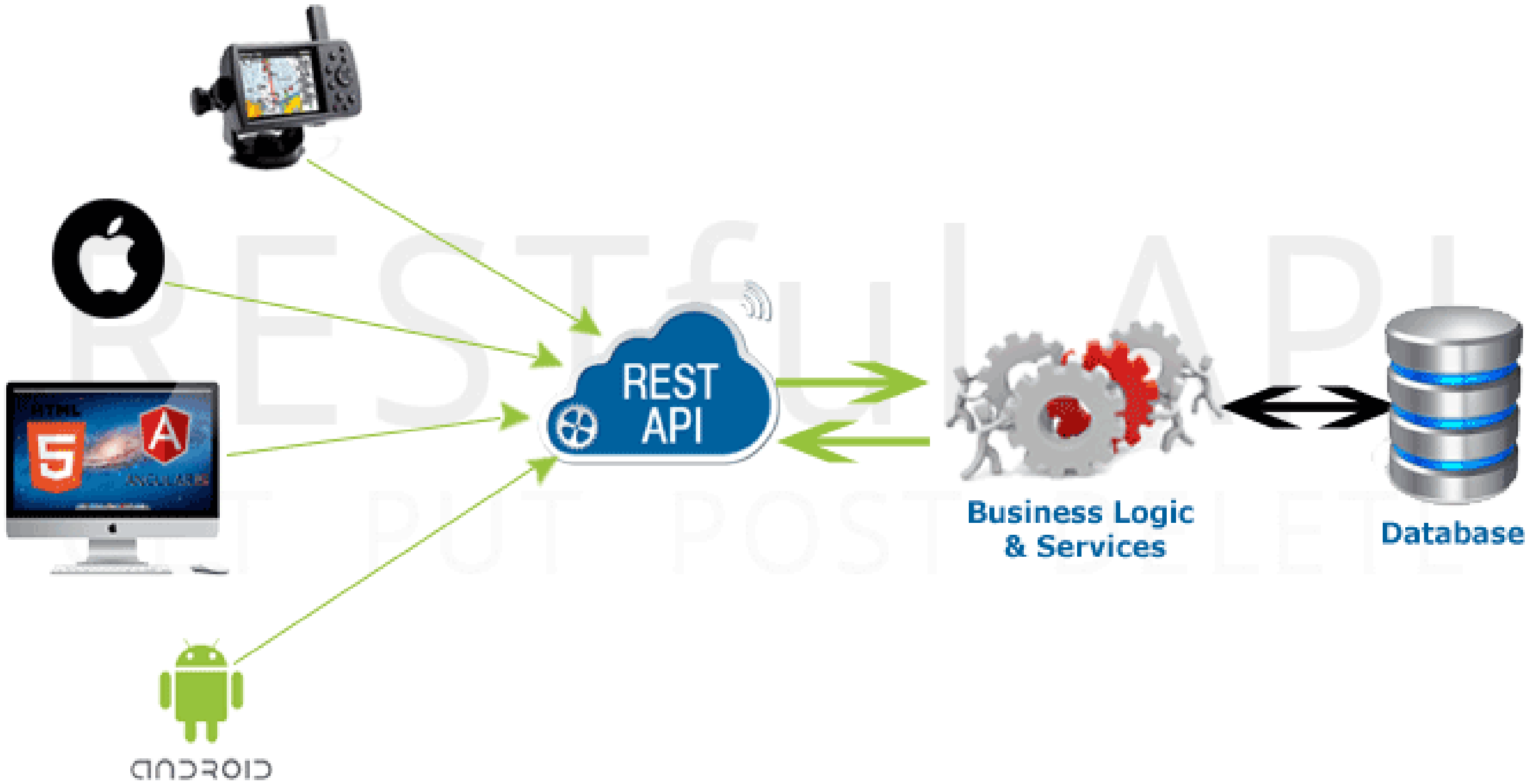
Distributed System Example Scenarios



Fetch Real World API



Fetch Machine Learning Model via API



Introduction

REST stands for **RE**presentational **S**tate **T**ransfer.

API stands for **A**pplication **P**rogramming **I**nterface.

REST API allows applications to interact with each other and exchange data.



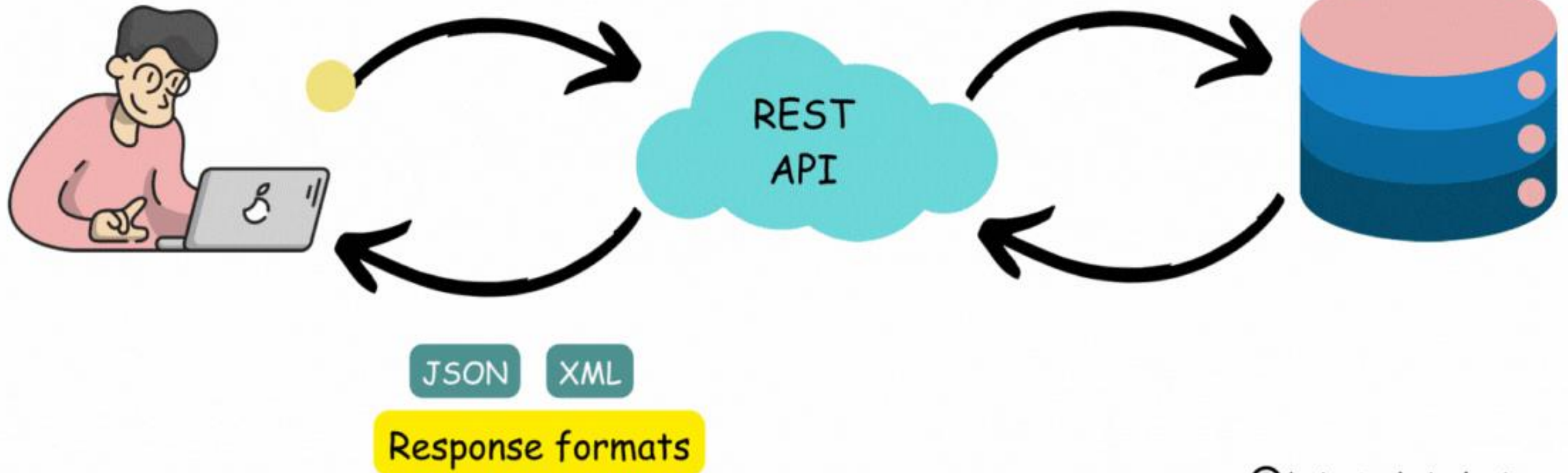
HTTP Methods

Create (POST)

Read (GET)

Update (PUT)

DELETE



This lecture will
cover only GET
operation



HTTP Verbs

- Post
- Get
- Put or Patch
- Delete

CRUD Operations

- Create
- Read
- Update
- Delete

URI	HTTP Verb	Outcome
.../api/employees	GET	Gets list of employees
.../api/employees/1	GET	Gets employee with Id = 1
.../api/employees	POST	Creates a new employee
.../api/employees/1	PUT	Updates employee with Id = 1
.../api/employees/1	DELETE	Deletes employee with Id = 1

REST API using Flask



+



Flask

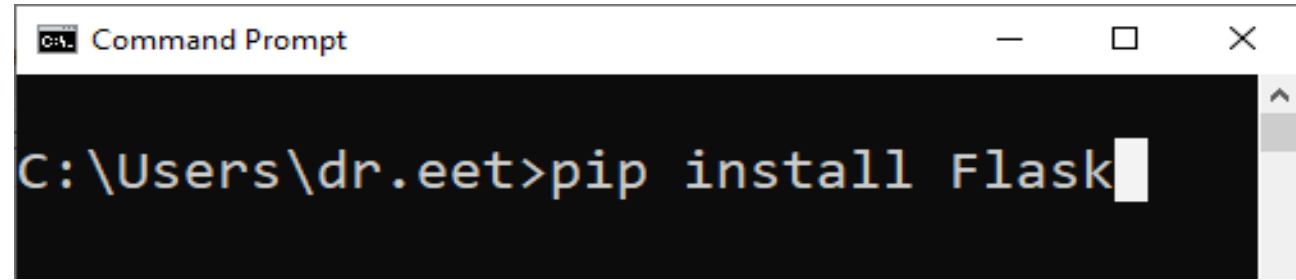
Python Flask Example

Flask is a lightweight and powerful web framework (micro-framework) for Python.

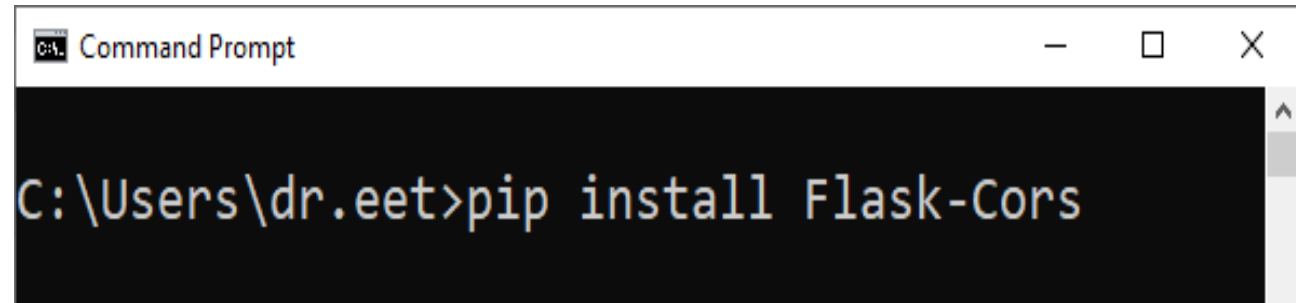
Flash functions on the principle of a Web Server Gateway Interface (WSGI).

Flask allows Python developers to create lightweight REST APIs.

Flask-CORS is a Flask extension that enables Cross-Origin Resource Sharing support for Flask applications.



```
Command Prompt
C:\Users\dr.eet>pip install Flask
```



```
Command Prompt
C:\Users\dr.eet>pip install Flask-Cors
```


Python Flask Example

```
from flask import Flask, request, render_template_string

app=Flask(__name__)

# HTML template
html_template = '''
<!DOCTYPE html>
<html>
<head><title>Simple Flask App</title></head>
<body>
<h1>Flask Test</h1>
  <h2>Enter Your Name</h2>
  <form method="POST">
    <input type="text" name="username">
    <input type="submit" value="Submit">
  </form>

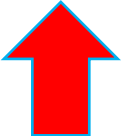
  {% if name %}
    <h3>Hello, {{ name }}!</h3>
  {% endif %}
</body>
</html>
...
'''
```

Python Flask Example

```
@app.route('/', methods=['GET', 'POST'])
def home():
    name=None
    if request.method=='POST':
        name=request.form.get('username')
    return render_template_string(html_template, name=name)

def run_app():
    app.run("172.21.13.167",port=5000)

#import threading
#thread=threading.Thread(target=run_app)
#thread.start()
```



Example Scenarios for Simple Distributed System

1. Client-Server Architecture in Distributed System

Python API + JavaScript frontend (browser on another server - Tomcat)

Python Flask serves /api/students.

JavaScript (React/HTML on another server) fetches that API.

2. Multi-service Distributed System

Python provides JSON data.

PHP fetches that API and renders HTML.

two backend servers working together

3. Microservices-based Distributed System (IPC)

Microservice A (KPay) manages user wallet balances

Microservice B (Atom Telecom) manages telecom account balances

Microservice C (Consumer) coordinates the workflow of payments

1. Client-Server Architecture in Distributed System (Python API + JS on Tomcat)

```
from flask import Flask, jsonify
from flask_cors import CORS # To allow cross-origin requests

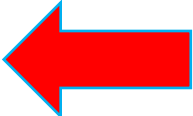
app = Flask(__name__)
CORS(app)
# Enable CORS so other servers (like your HTML frontend) can access it
# Cross-Origin Resource Sharing (CORS) extension or middleware

#student data
students = [
    {"id": 1, "name": "Rose", "age": 20, "grade": "A"},
    {"id": 2, "name": "Jiso", "age": 22, "grade": "B"},
    {"id": 3, "name": "Lisa", "age": 21, "grade": "A-"},
]

#function index()
@app.route("/")
def index():
    return jsonify(students)

# API endpoint
@app.route('/api/students', methods=['GET'])
def get_students():
    return jsonify(students)

from werkzeug.serving import run_simple
run_simple("172.21.13.167", 8282, app, use_reloader=False)
```



1. Client-Server Architecture in Distributed System (Python API + JS on Tomcat)

Create Dynamic Web Project with
Eclipse IDE and Tomcat Server

index.html X

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Student Data</title>
5 </head>
6 <body>
7   <h1>Student List</h1>
8   <ul id="student-list"></ul>
9
10  <script>
11    // Change URL to your Python server IP/domain
12    fetch("http://172.21.13.167:8282/api/students")
13      .then(response => response.json())
14      .then(data => {
15        const list = document.getElementById("student-list");
16        data.forEach(student => {
17          const item = document.createElement("li");
18          item.textContent = `${student.name} (Age: ${student.age}, Grade: ${student.grade})`;
19          list.appendChild(item);
20        });
21      })
22      .catch(error => console.error("Error fetching student data:", error));
23  </script>
24 </body>
25 </html>
```

2. Multi-service Distributed System (Python API + PHP on Apache)

index.php X

index.php > html

```
1  <?php
2  // API endpoint (adjust host/IP as needed)
3  $api_url = "http://172.21.13.167:8282/api/students";
4
5  // Fetch JSON data from Python API
6  $json_data = file_get_contents($api_url);
7
8  // Decode JSON into PHP array
9  $students = json_decode($json_data, true);
10 ?>
11 <!DOCTYPE html>
12 <html>
13 <head>
14 |   <title>Student Data from Python API</title>
15 </head>
16 <body>
17 |   <h1>Student List</h1>
18 |   <ul>
19 |       <?php foreach ($students as $student): ?>
20 |           <li>
21 |               <?php echo htmlspecialchars($student['name']); ?>
22 |               (Age: <?php echo htmlspecialchars($student['age']); ?>,
23 |               Grade: <?php echo htmlspecialchars($student['grade']); ?>)
24 |           </li>
25 |       <?php endforeach; ?>
26 |   </ul>
27 </body>
28 </html>
```

Create PHP Web Project with
VSCode and Apache Server

3. Microservices Architecture and Interprocess Communication in a Distributed System

Microservice A (KPay) manages user wallet balances

Microservice B (Atom Telecom) manages telecom account balances

Microservice C (Consumer) coordinates the workflow of payments

Each service has:

its own data (separate database)

its own API

runs independently on different ports (7001, 7002, 7003)

Interprocess Communication (IPC): Services are separate processes and can run on different servers.

User – HTML in Service C - /pay-bill

Service C (Consumer) - /pay

Service A (KPay)

- Deducts KPay balance
- Returns new KPay balance → Service C
- Also calls Service B (/add)

Service B (Atom)

- Updates its balance
- Returns new Telecom balance → Service C

Service C

- Shows both balances (KPay + Telecom) to **User**

All data exchange is in **JSON** over HTTP.

```
from flask import Flask, request, jsonify
import requests
from flask_cors import CORS
```

```
app = Flask(__name__)
CORS(app)
```

```
kpay_balances = {1: 20000, 2: 15000, 3: 10000}
TELECOM_URL = "http://172.21.13.167:7002"
```

```
@app.route('/pay', methods=['POST'])
```

```
def process_payment():
```

```
    data = request.get_json()
```

```
    user_id = data.get("user_id")
```

```
    amount = int(data.get("amount", 0))
```

```
    if user_id not in kpay_balances:
```

```
        return jsonify({"error": "User not found"}), 404
```

```
    if kpay_balances[user_id] < amount:
```

```
        return jsonify({"status": "failed", "message": "Insufficient balance"}), 400
```

```
    # Deduct balance
```

```
    kpay_balances[user_id] -= amount
```

```
    # Trigger telecom update (but Service B reports back to C, not to A)
```

```
    telecom_payload = {"user_id": user_id, "amount": amount}
```

```
    requests.post(f"{TELECOM_URL}/add", json=telecom_payload)
```

```
    # Return only KPay balance
```

```
    return jsonify({
```

```
        "status": "success",
```

```
        "message": f"Deducted {amount} from KPay for user {user_id}",
```

```
        "user_id": user_id,
```

```
        "kpay_balance": kpay_balances[user_id]
```

```
    })
```

```
from werkzeug.serving import run_simple
```

```
run_simple("172.21.13.167", 7001, app, use_reloader=False)
```

Microservice A (KPay) Synchronous


```
from flask import Flask, request, jsonify
import requests
from flask_cors import CORS
```

```
app = Flask(__name__)
CORS(app)
```

```
telecom_balances = {1: 1000, 2: 3000, 3: 0}
CONSUMER_URL = "http://172.21.13.167:7003"
```

```
@app.route('/add', methods=['POST'])
```

```
def add_balance():
```

```
    data = request.get_json()
```

```
    user_id = data.get("user_id")
```

```
    amount = int(data.get("amount", 0))
```

```
    if user_id not in telecom_balances:
```

```
        return jsonify({"error": "User not found"}), 404
```

```
    # Add to telecom balance
```

```
    telecom_balances[user_id] += amount
```

```
    # Inform Service C (Consumer) about new balance
```

```
    notify_payload = {
```

```
        "user_id": user_id,
```

```
        "telecom_balance": telecom_balances[user_id]
```

```
    }
```

```
    try:
```

```
        requests.post(f"{CONSUMER_URL}/notify-telecom", json=notify_payload)
```

```
    except Exception as e:
```

```
        print("Failed to notify Consumer:", str(e))
```

```
    return jsonify({
```

```
        "status": "success",
```

```
        "user_id": user_id,
```

```
        "telecom_balance": telecom_balances[user_id]
```

```
    })
```

```
from werkzeug.serving import run_simple
```

```
run_simple("172.21.13.167", 7002, app, use_reloader=False)
```

Microservice B (Atom) Synchronous

```

from flask import Flask, request, render_template_string
import requests
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

KPAY_URL = "http://172.21.13.167:7001"

# Store balances in memory for demo
last_result = {}

HTML_PAGE = """
<!DOCTYPE html>
<html>
<head>
    <title>Pay Telecom Bill</title>
</head>
<body>
    <h1>Pay Atom Bill via KPay</h1>
    <form method="POST" action="/pay-bill">
        <label>User ID:</label>
        <input type="number" name="user_id" required><br><br>
        <label>Amount:</label>
        <input type="number" name="amount" required><br><br>
        <button type="submit">Pay Now</button>
    </form>

    {% if result %}
        <h2>Payment Result</h2>
        {% if result.get("message") %}<p>{{ result['message'] }}</p>{% endif %}
        {% if result.get("kpay_balance") %}<p><b>KPay Balance:</b> {{ result['kpay_balance'] }}</p>{%
        {% if result.get("telecom_balance") %}<p><b>Telecom Balance:</b> {{ result['telecom_balance'] }}
    {% endif %}
</body>
</html>
"""

@app.route('/')
def home():
    return render_template_string(HTML_PAGE, result=last_result)

```

Microservice C (User) Synchronous

```
@app.route('/')
def home():
    return render_template_string(HTML_PAGE, result=last_result)
```

```
@app.route('/pay-bill', methods=['POST'])
```

```
def pay_bill():
    import time
    user_id = int(request.form.get("user_id"))
    amount = int(request.form.get("amount"))

    # Call KPay
    payload = {"user_id": user_id, "amount": amount}
    resp = requests.post(f"{KPAY_URL}/pay", json=payload)
    last_result.update(resp.json()) # Update with KPay response

    # Short wait so Telecom can notify
    time.sleep(1)
    return render_template_string(HTML_PAGE, result=last_result)
```

```
@app.route('/notify-telecom', methods=['POST'])
```

```
def notify_telecom():
    data = request.get_json()
    last_result.update(data) # Store telecom balance
    return {"status": "received"}
```

```
from werkzeug.serving import run_simple
run_simple("172.21.13.167", 7003, app, use_reloader=False)
```

Microservice C (User)
Synchronous

Synchronous Blocking Issues

1. Service C (/pay-bill) calls Service A /pay.
2. Service A /pay calls Service B /add synchronously.
3. Service B /add calls Service C /notify-telecom synchronously.
4. Meanwhile, Service C is still waiting for the response from Service A.

This creates a circular wait / blocking call:

- Service C waits for Service A →
- Service A waits for Service B →
- Service B waits for Service C →

Deadlock / request hangs.

Fix Issues with Asynchronous Non-Blocking

- Service B updates its internal balance.
- Instead of directly calling Service C, Service C polls or fetches the balance after payment, or uses a message queue / webhook.

Remove the circular call

1. Let Service C only call Service A.
2. Service A calls Service B.
3. Service A collects both KPay and Atom balances and returns them to Service C.
4. Service B no longer calls Service C directly.

This is simpler and avoids hanging.

```
from flask import Flask, request, jsonify
import requests
from flask_cors import CORS
```

```
app = Flask(__name__)
CORS(app)
```

```
kpay_balances = {1: 20000, 2: 15000, 3: 10000}
TELECOM_URL = "http://172.21.13.167:7002"
```

```
@app.route('/pay', methods=['POST'])
```

```
def process_payment():
```

```
    data = request.get_json()
```

```
    user_id = data.get("user_id")
```

```
    amount = int(data.get("amount", 0))
```

```
    # Deduct KPay
```

```
    kpay_balances[user_id] -= amount
```

```
    # Call Telecom service
```

```
    telecom_payload = {"user_id": user_id, "amount": amount}
```

```
    tel_resp = requests.post(f"{TELECOM_URL}/add", json=telecom_payload)
```

```
    tel_balance = tel_resp.json()["telecom_balance"]
```

```
    return jsonify({
```

```
        "status": "success",
```

```
        "message": f"Payment of {amount} completed for user {user_id}",
```

```
        "kpay_balance": kpay_balances[user_id],
```

```
        "telecom_balance": tel_balance
```

```
    })
```

```
from werkzeug.serving import run_simple
```

```
run_simple("172.21.13.167", 7001, app, use_reloader=False)
```

Microservice A (KPay) Asynchronous

```
from flask import Flask, request, jsonify
import requests
from flask_cors import CORS
```

Microservice B (Atom) Asynchronous

```
app = Flask(__name__)
CORS(app)
```

```
telecom_balances = {1: 1000, 2: 3000, 3: 0}
CONSUMER_URL = "http://172.21.13.167:7003"
```

```
@app.route('/add', methods=['POST'])
def add_balance():
    data = request.get_json()
    user_id = data.get("user_id")
    amount = int(data.get("amount", 0))
    telecom_balances[user_id] += amount
    return jsonify({
        "status": "success",
        "telecom_balance": telecom_balances[user_id]
    })
```

```
from werkzeug.serving import run_simple
run_simple("172.21.13.167", 7002, app, use_reloader=False)
```

```
from flask import Flask, request, render_template_string
import requests
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

KPAY_URL = "http://172.21.13.167:7001"

# Store balances in memory for demo
last_result = {}

HTML_PAGE = """
<!DOCTYPE html>
<html>
<head>
    <title>Pay Telecom Bill</title>
</head>
<body>
    <h1>Pay Atom Bill via KPay</h1>
    <form method="POST" action="/pay-bill">
        <label>User ID:</label>
        <input type="number" name="user_id" required><br><br>

        <label>Amount:</label>
        <input type="number" name="amount" required><br><br>

        <button type="submit">Pay Now</button>
    </form>

    {% if result %}
        <h2>Payment Result</h2>
        {% if result.get("message") %}<p>{{ result['message'] }}</p>{% endif %}
        {% if result.get("kpay_balance") %}<p><b>KPay Balance:</b> {{ result['kpay_balance'] }}</p>{%
        {% if result.get("telecom_balance") %}<p><b>Telecom Balance:</b> {{ result['telecom_balance']
    {% endif %}
</body>
</html>
"""
```

Microservice C (User) Asynchronous


```
@app.route('/')
def home():
    return render_template_string(HTML_PAGE, result=last_result)
```

```
@app.route('/pay-bill', methods=['POST'])
def pay_bill():
    user_id = int(request.form.get("user_id"))
    amount = int(request.form.get("amount"))
```

Microservice C (User) Asynchronous

```
    # Call KPay service (Service A)
    payload = {"user_id": user_id, "amount": amount}
    resp = requests.post(f"{KPAY_URL}/pay", json=payload)
    result = resp.json() # This already includes telecom balance from Service A
    return render_template_string(HTML_PAGE, result=result)
```

```
from werkzeug.serving import run_simple
run_simple("172.21.13.167", 7003, app, use_reloader=False)
```

1

Model Training



2

Create Flask REST
API



Flask

3

Predict result



Restful API endpoint in Flask

Home

KNN_Model

KNN_API

localhost:8888/tree/ModelAPI/KNN_Color

jupyter

FileViewSettingsHelp

FilesRunning

Select items to perform actions on them.

NewUpload

/ ModelAPI / KNN_Color /

☐ Name	Modified	File Size
☐ templates	35 minutes ago	
☒ KNN_API.ipynb	27 minutes ago	1.9 KB
☒ KNN_Model.ipynb	27 minutes ago	1.8 KB
☐ color.csv	last year	110 B
☐ knnModel.pkl	2 months ago	779 B

Serialize and Deserialize Object using Pickle

The pickle module in Python provides a way to **serialize and deserialize** Python object structures.

Serialization (Pickling)

- Conversion to Byte Stream:
`pickle.dump()`
- Handling Complex Objects
- Storage or Transmission

Deserialization (Unpickling)

- Byte Stream to Object: **`pickle.load()`**
- Object Reconstruction

Pickle is designed specifically for Python objects, pickle data generally cannot be directly exchanged with other programming languages.

Fetch Machine Learning Model via API

KNN Model Building

```
import numpy as np
import pandas as pd
import pickle

#Load Dataset
data_set=pd.read_csv('color.csv')

#Split Features and Target
X=data_set[['Brightness','Saturation']]
y=data_set[['Class']]

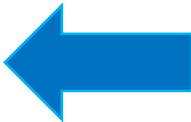
#Split Training and Testing
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test=train_test_split(X,y,random_state=0)

#Train KNN Model
from sklearn.neighbors import KNeighborsClassifier
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train.values,y_train.values.ravel())

#Predict
y_predict=knn.predict(X_test)

#Accuracy
from sklearn.metrics import accuracy_score
print("Accuracy Score >> ", accuracy_score(y_test , y_predict)*100)

#Save Model using Pickle
with open("knnModel.pkl", "wb") as model_file:
    pickle.dump(knn, model_file)
```



`pickle.dump( )`

Serialization

```
import numpy as np
import pickle
from flask import Flask, render_template, request
```

```
#Create instance of the class
app=Flask(__name__)
```

```
#function index()
@app.route("/")
def index():
    return render_template("index.html")
```

```
#prediction function
```

```
def prediction(features_list):
    features=np.array(features_list).reshape(1,2)
    load_model=pickle.load(open('knnModel.pkl','rb'))
    target=load_model.predict(features)
    return target[0]
```

```
@app.route('/result', methods=['POST'])
def result():
    if request.method=='POST':
        features_list=request.form.to_dict()
        features_list=list(features_list.values())
        features_list=list(map(int, features_list))
        target=prediction(features_list)
        return render_template("result.html", target=target)
```

```
from werkzeug.serving import run_simple
run_simple("0.0.0.0", 5000, app, use_reloader=False)
```

Fetch Machine Learning
Model via API

KNN Model API

Deserialization

 pickle.load()

```
<html>
<body>
  <h3>Color Feature</h3>
```

index.html

```
<div>
  <form action="/result" method="POST">

    <label for="brightness">Brightness </label>
    <input type="text" id="brightness" name="brightness">
    <br>

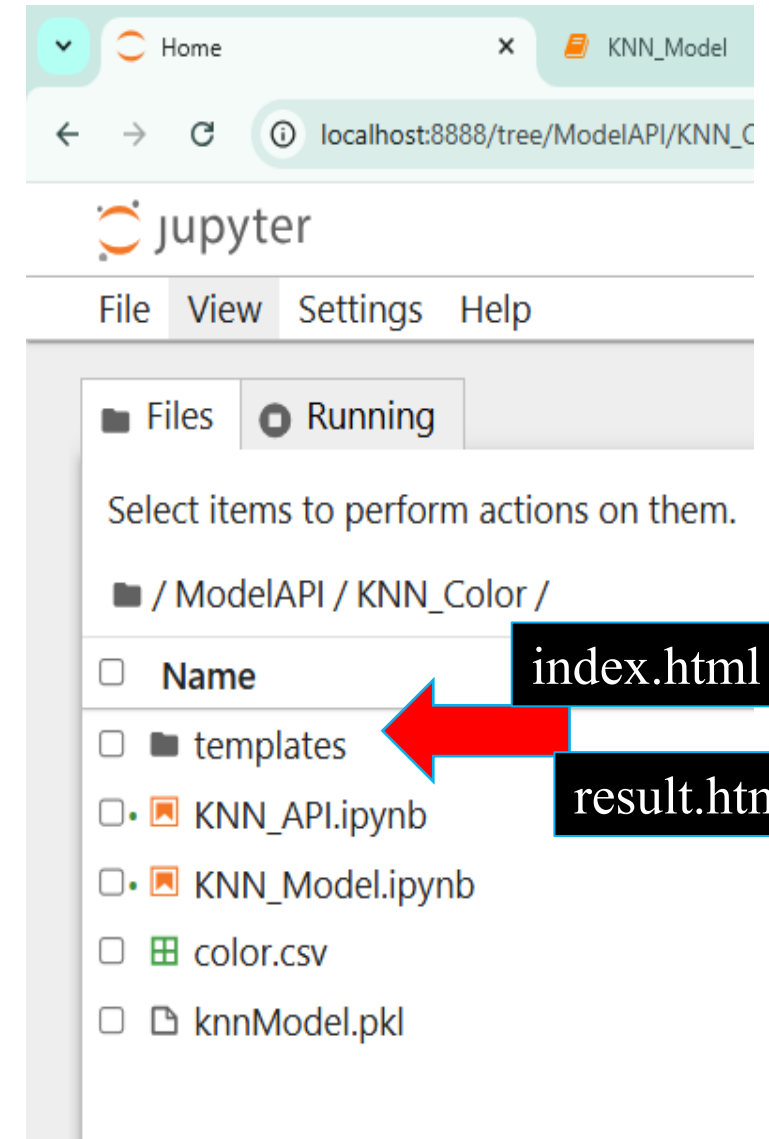
    <label for="saturation">Saturation </label>
    <input type="text" id="saturation" name="saturation">
    <br>

    <br>
    <input type="submit" value="Submit">
  </form>
</div>
</body>
</html>
```

```
<!doctype html>
<html>
  <body>
    <h1> {{ target }}</h1>
  </body>
</html>
```

result.html

Access Model API via HTML UI



Accessing or Fetch Real World API

```
[1]: import requests

resp = requests.get("https://catfact.ninja/fact", headers={"User-Agent": "example"}, timeout=10)
resp.raise_for_status()
data = resp.json()
print(data["fact"])
```

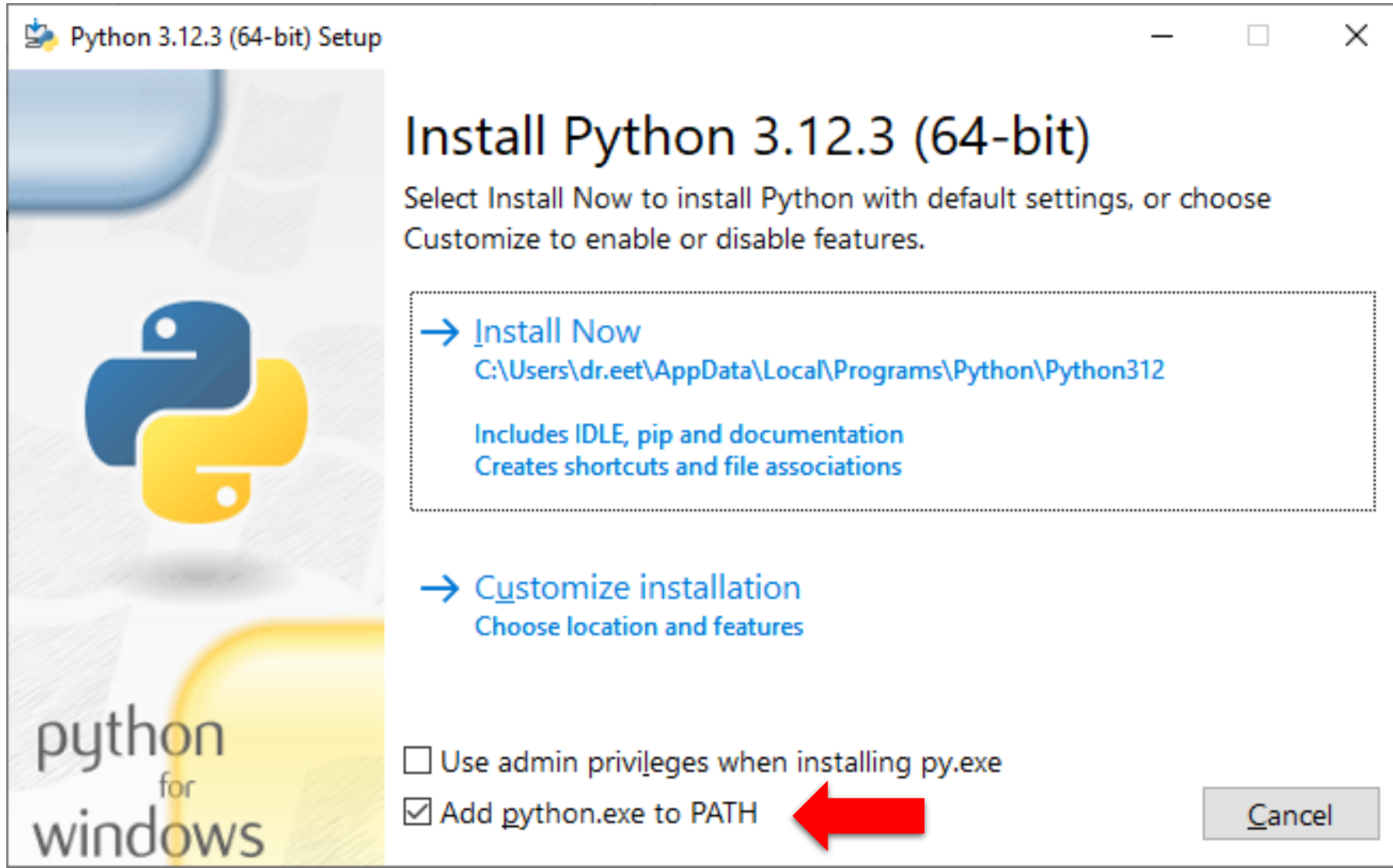
Cats are North America's most popular pets: there are 73 million cats compared to 63 million dogs. Over 30% of households in North America own a cat.

Configuring Jupyter notebook for Python

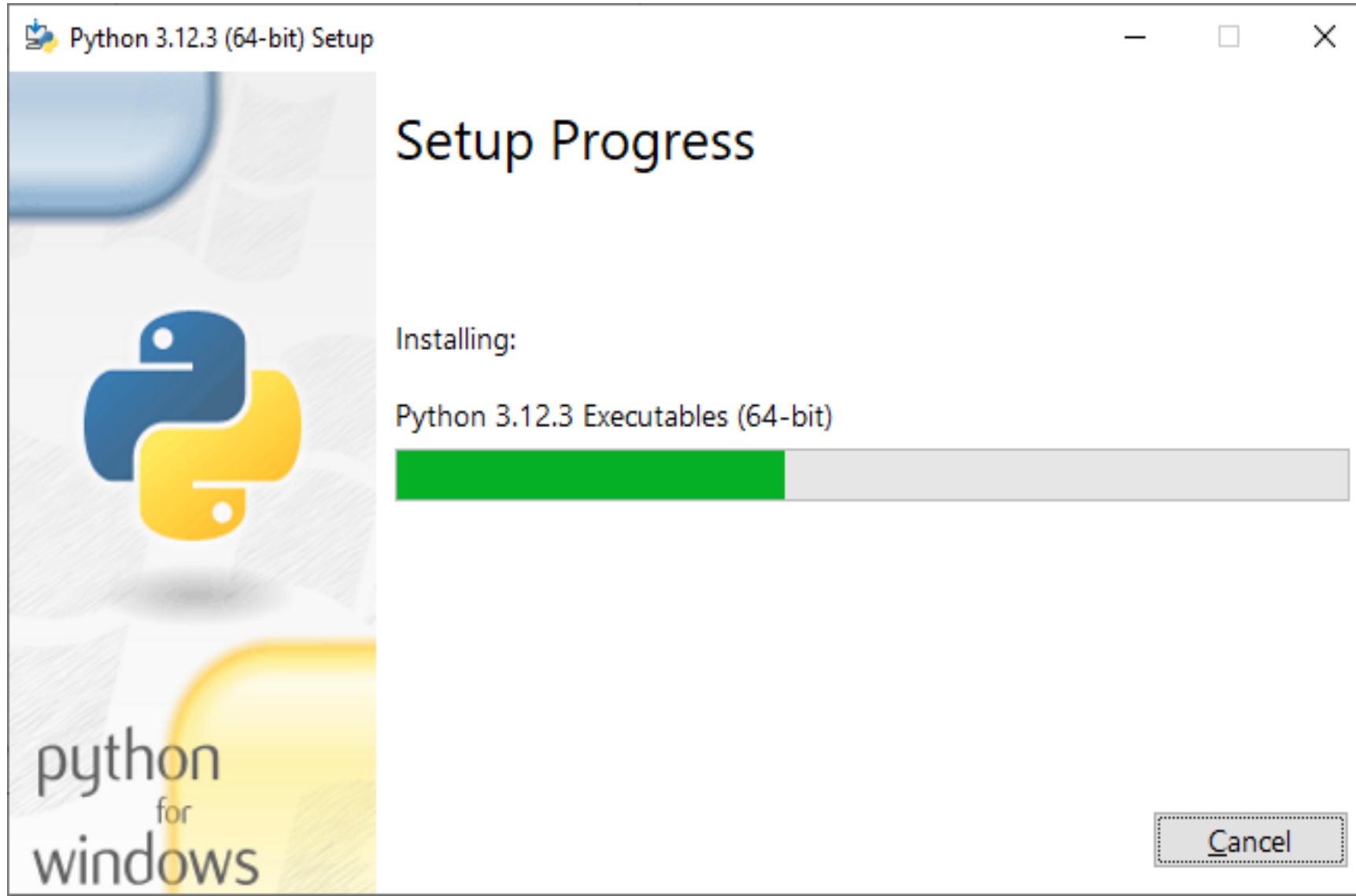


Step 1: Install Python

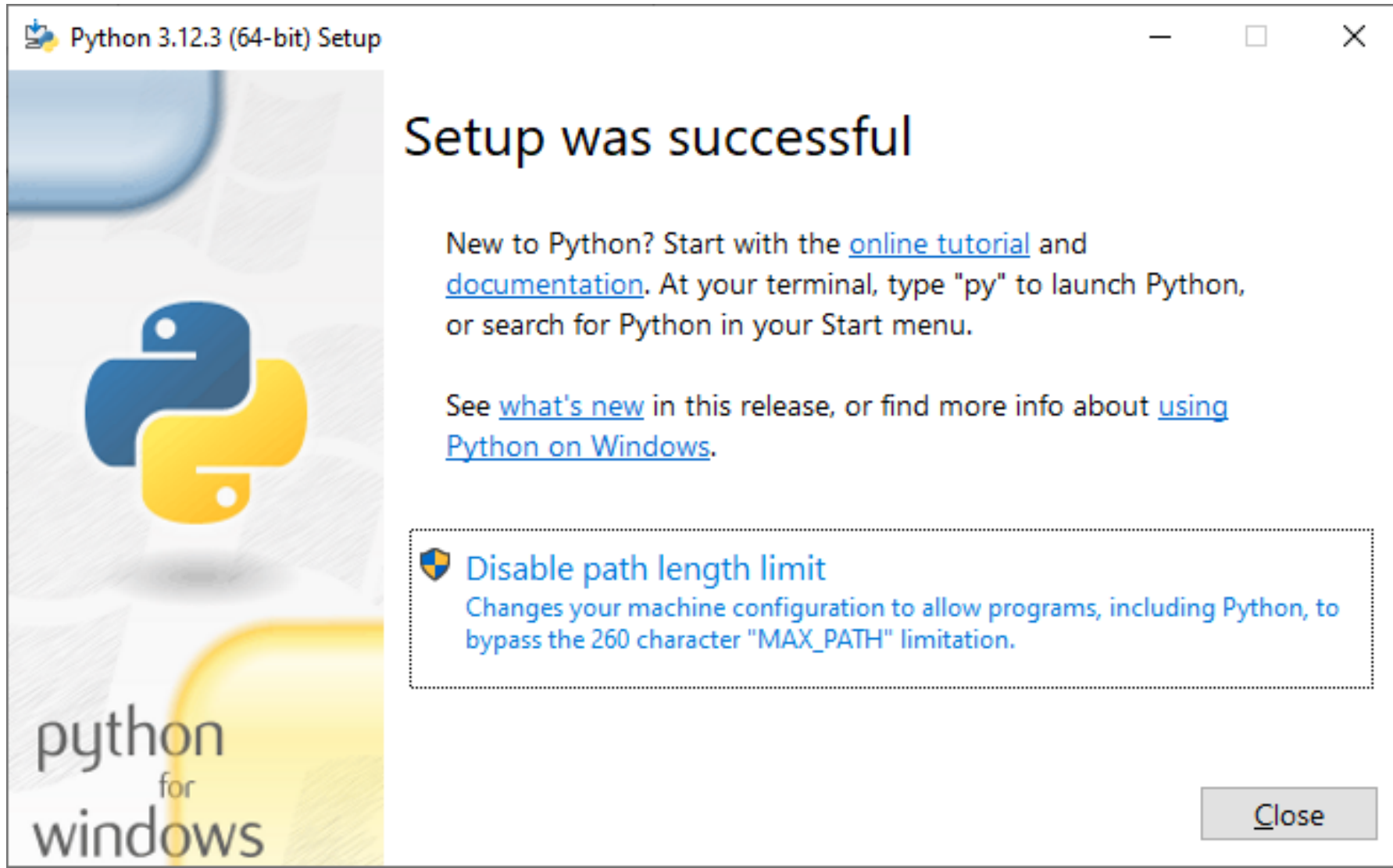
<https://www.python.org/downloads/>



Step 1: Install Python -> Setup Progress



Step 1: Install Python -> Setup was Successful



Step 2: Install Jupyter Notebook via PIP



```
Command Prompt

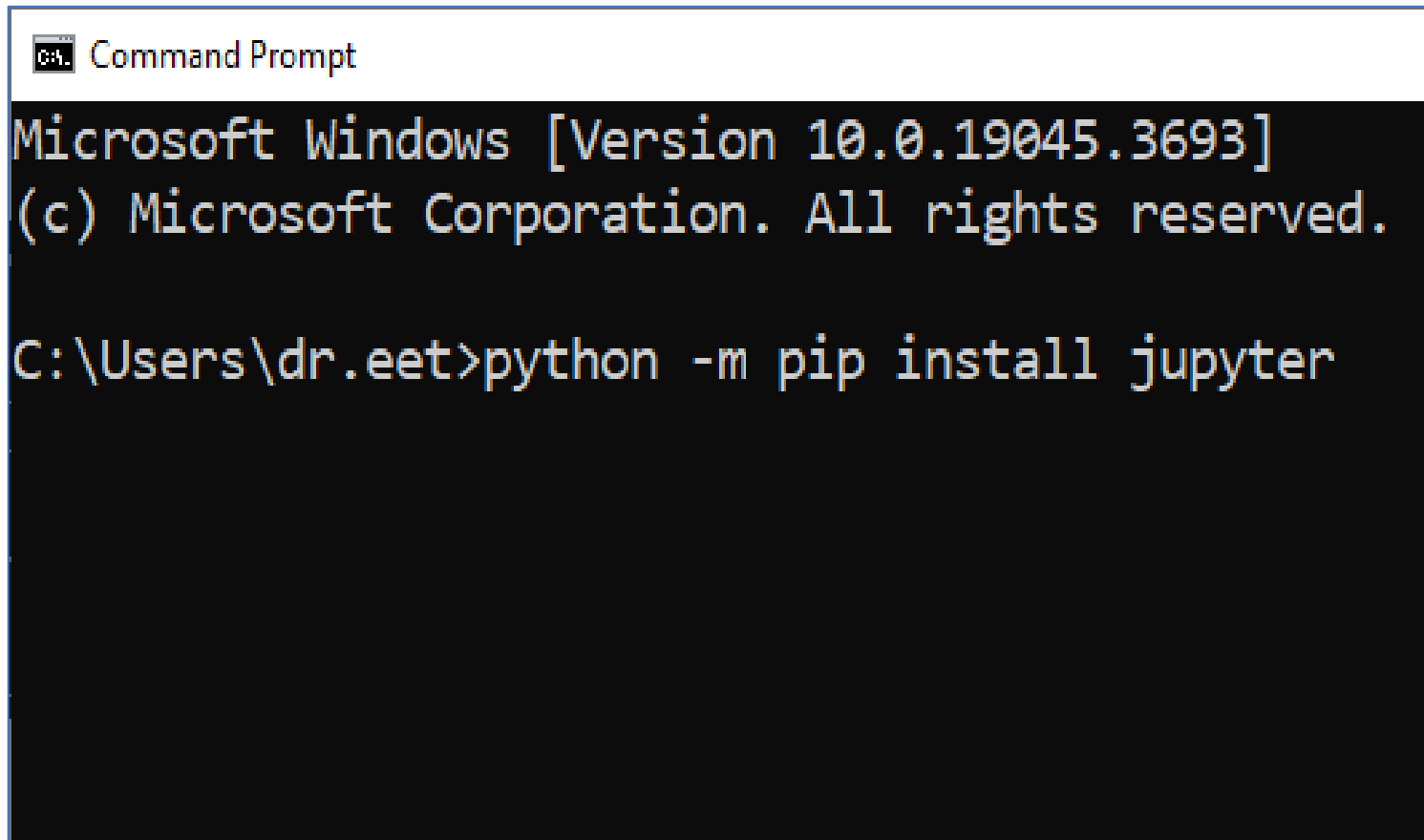
C:\Users\dr.eet>python --version
Python 3.12.3

C:\Users\dr.eet>
```

1. Open Command Prompt by searching 'cmd' in the Windows search bar.
2. Check if Python is installed by running the 'python --version' command.

Step 2: Install Jupyter Notebook via PIP

1. To install Jupyter Notebook via pip, run the command
'**python -m pip install jupyter**'

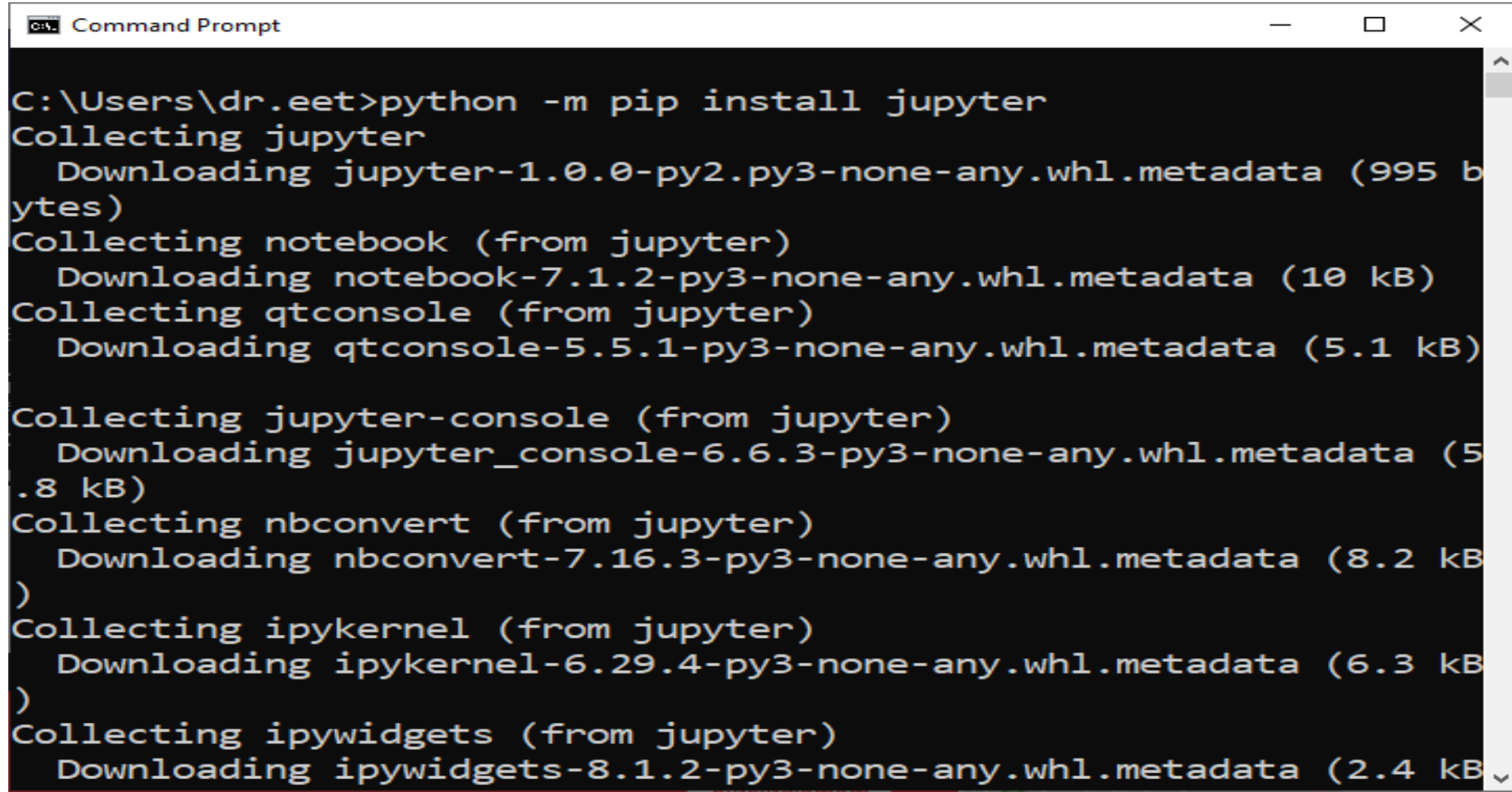
A screenshot of a Windows Command Prompt window. The title bar says 'C:\ Command Prompt'. The text inside shows the Windows version and copyright information, followed by the command 'python -m pip install jupyter' being entered at the prompt.

```
C:\ Command Prompt
Microsoft Windows [Version 10.0.19045.3693]
(c) Microsoft Corporation. All rights reserved.

C:\Users\dr.eet>python -m pip install jupyter
```

Step 2: Install Jupyter Notebook -> Installation Progress

1. Please wait to complete the installation process

A screenshot of a Windows Command Prompt window titled 'Command Prompt'. The window has a black background with white text. The text shows the command 'python -m pip install jupyter' being executed. The output shows the installation progress for various packages: jupyter, notebook, qtconsole, jupyter-console, nbconvert, ipykernel, and ipywidgets. Each package is shown with its version and the size of the metadata file being downloaded.

```
C:\Users\dr.eet>python -m pip install jupyter
Collecting jupyter
  Downloading jupyter-1.0.0-py2.py3-none-any.whl.metadata (995 b
ytes)
Collecting notebook (from jupyter)
  Downloading notebook-7.1.2-py3-none-any.whl.metadata (10 kB)
Collecting qtconsole (from jupyter)
  Downloading qtconsole-5.5.1-py3-none-any.whl.metadata (5.1 kB)

Collecting jupyter-console (from jupyter)
  Downloading jupyter_console-6.6.3-py3-none-any.whl.metadata (5
.8 kB)
Collecting nbconvert (from jupyter)
  Downloading nbconvert-7.16.3-py3-none-any.whl.metadata (8.2 kB
)
Collecting ipykernel (from jupyter)
  Downloading ipykernel-6.29.4-py3-none-any.whl.metadata (6.3 kB
)
Collecting ipywidgets (from jupyter)
  Downloading ipywidgets-8.1.2-py3-none-any.whl.metadata (2.4 kB
```

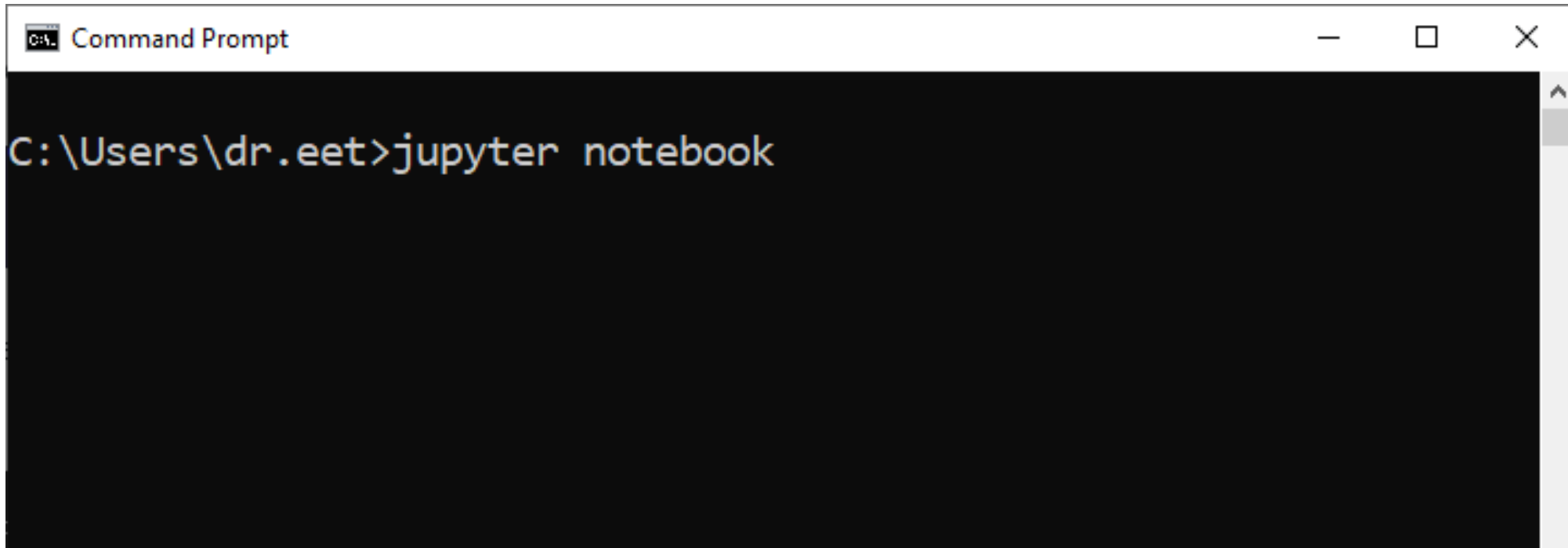
Step 2: Install Jupyter Notebook -> Installation Success

```
Command Prompt
une-3.0.2 nbclient-0.10.0 nbconvert-7.16.3 nbformat-5.10.4 nest-
asyncio-1.6.0 notebook-7.1.2 notebook-shim-0.2.4 overrides-7.7.0
packaging-24.0 pandocfilters-1.5.1 parso-0.8.4 platformdirs-4.2
.0 prometheus-client-0.20.0 prompt-toolkit-3.0.43 psutil-5.9.8 p
ure-eval-0.2.2 pycparser-2.22 pygments-2.17.2 python-dateutil-2.
9.0.post0 python-json-logger-2.0.7 pywin32-306 pywinpty-2.0.13 p
yyaml-6.0.1 pyzmq-26.0.0 qtconsole-5.5.1 qtpy-2.4.1 referencing-
0.34.0 requests-2.31.0 rfc3339-validator-0.1.4 rfc3986-validator
-0.1.1 rpds-py-0.18.0 send2trash-1.8.3 six-1.16.0 sniffio-1.3.1
soupsieve-2.5 stack-data-0.6.3 terminado-0.18.1 tinycss2-1.2.1 t
ornado-6.4 traitlets-5.14.2 types-python-dateutil-2.9.0.20240316
uri-template-1.3.0 urllib3-2.2.1 wcwidth-0.2.13 webcolors-1.13
webencodings-0.5.1 websocket-client-1.7.0 widgetsnbextension-4.0
.10

C:\Users\dr.eet>
```


Step 3: Access Jupyter Notebook

To access Jupyter Notebook via command prompt, run the command
‘jupyter notebook’

A screenshot of a Windows Command Prompt window. The title bar reads 'C:\> Command Prompt'. The command prompt shows the path 'C:\Users\dr.eet>' followed by the command 'jupyter notebook' entered in white text on a black background.

နောက်တစ်ခု Jupyter Notebook ပြန်ဖွင့်ချင်တိုင်း Command Prompt ကို ဖွင့်ပြီး ဒီနေရာကနေပြန်စရပါမယ်
ရှေ့က Installation Process တွေ ပြန်လုပ်စရာမလိုပါ

The Jupyter Notebook will open via Browser.

A screenshot of a web browser window displaying the Jupyter Notebook interface. The browser's address bar shows 'localhost:8888/tree'. The page header includes the Jupyter logo and navigation links: 'File', 'View', 'Settings', and 'Help'. Below the header, there are two tabs: 'Files' and 'Running'. The 'Files' tab is active, showing a file tree view. At the top of the file tree, there is a prompt 'Select items to perform actions on them.' and three buttons: 'New', 'Upload', and a refresh icon. The file tree lists various system folders, each with a checkbox to its left. The 'Last Modified' column shows the time since the last modification for each folder. The 'File Size' column is present but empty for these folders.

☐ Name	Last Modified	File Size
☐ 3D Objects	4 months ago	
☐ Contacts	4 months ago	
☐ Desktop	20 days ago	
☐ Documents	29 days ago	
☐ Downloads	21 minutes ago	
☐ Favorites	4 months ago	
☐ Links	4 months ago	
☐ Music	4 months ago	
☐ OneDrive	4 months ago	
☐ Pictures	4 months ago	
☐ Saved Games	4 months ago	
☐ Searches	4 months ago	
☐ Videos	4 months ago	

You can access the Python program file via Jupyter Notebook Workspace



The image shows a Windows File Explorer window on the left and a Jupyter Notebook interface on the right. A red arrow points from the File Explorer's address bar to the Jupyter interface.

Windows File Explorer (Left): The address bar shows the path: `This PC > Local Disk (C:) > Users > dr.eet`. The left sidebar shows the 'This PC' view with various folders. The main pane shows a list of files and folders in the user's home directory.

Name	Date modified
.cache	11/22/2023 11:20 A
.eclipse	12/28/2023 10:22 P
.ipython	4/17/2024 5:13 PM
.jupyter	4/17/2024 5:13 PM
.m2	11/30/2023 11:06 P
.ms-ad	11/22/2023 10:29 A
.p2	4/12/2024 1:49 PM
.vscode	11/24/2023 1:22 PM
3D Objects	11/22/2023 9:33 AM
Contacts	11/22/2023 9:33 AM
Desktop	3/27/2024 11:40 PM
Documents	3/19/2024 8:56 AM
Downloads	4/17/2024 4:52 PM
Favorites	11/22/2023 9:33 AM
Links	11/22/2023 9:33 AM
Music	11/22/2023 9:33 AM
OneDrive	11/22/2023 9:58 AM
Pictures	11/22/2023 9:35 AM
Saved Games	11/22/2023 9:33 AM
Searches	11/22/2023 9:35 AM
Videos	11/22/2023 9:30 PM

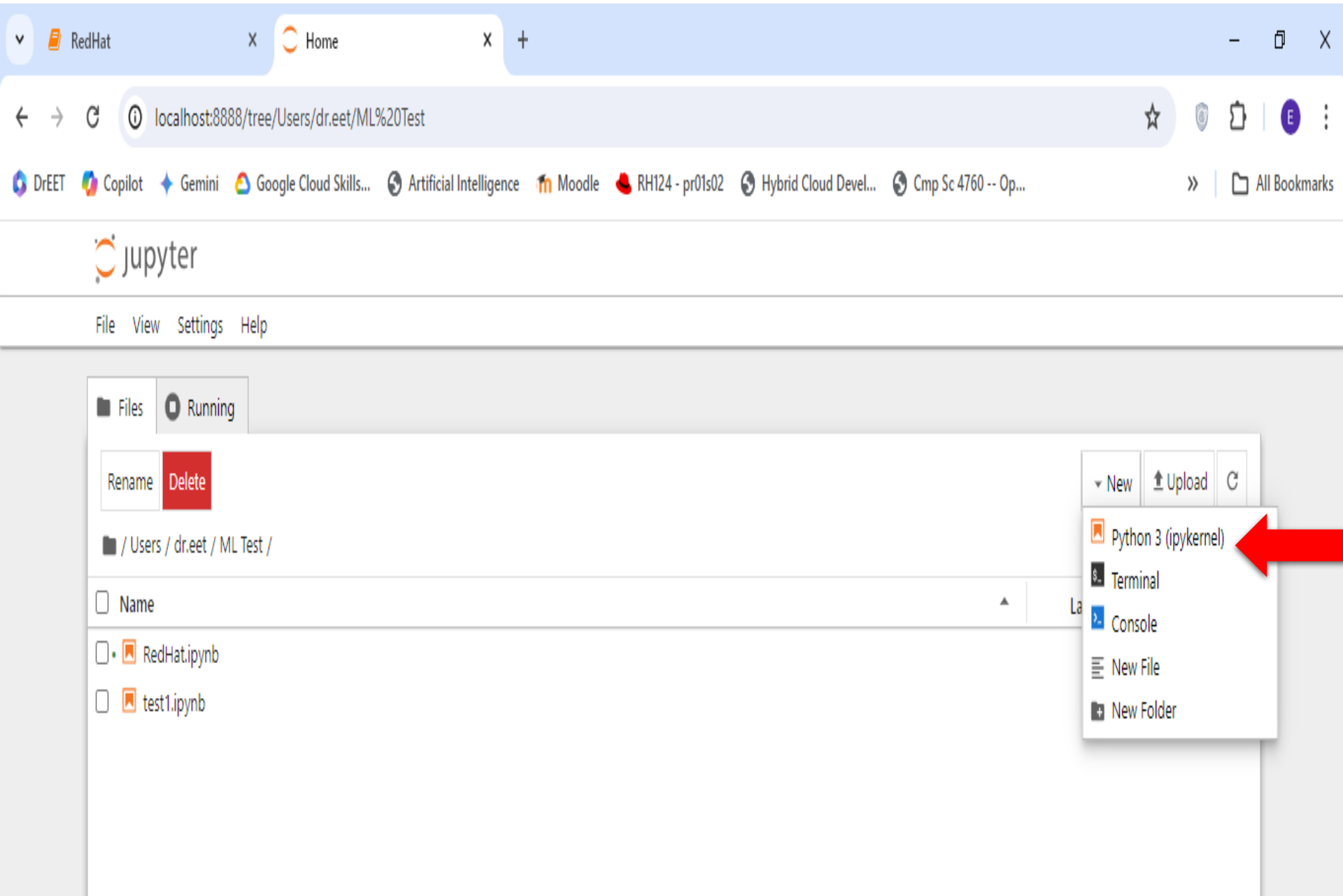
Jupyter Notebook Interface (Right): The browser address bar shows `localhost:8888/tree`. The Jupyter logo is visible. The 'Files' tab is selected, showing a tree view of the file system. The 'Running' tab is also visible. The 'Files' tab shows a list of files and folders in the current directory.

Name	Last Modified	File Size
3D Objects	4 months ago	
Contacts	4 months ago	
Desktop	20 days ago	
Documents	29 days ago	
Downloads	24 minutes ago	
Favorites	4 months ago	
Links	4 months ago	
Music	4 months ago	
OneDrive	4 months ago	
Pictures	4 months ago	
Saved Games	4 months ago	
Searches	4 months ago	
Videos	4 months ago	

Create a new folder to store Python programs.



A screenshot of the JupyterLab web interface. The browser's address bar shows 'localhost:8888/tree/Users/dr.eet'. The JupyterLab header includes the 'jupyter' logo and a menu bar with 'File', 'View', 'Settings', and 'Help'. Below the header, there are two tabs: 'Files' and 'Running'. The 'Files' tab is active, showing a file browser view of the directory '/ Users / dr.eet /'. A list of files and folders is displayed, including '3D Objects', 'Contacts', 'Desktop', 'Documents', 'Downloads', 'Favorites', 'Links', 'ML Test', 'Music', 'OneDrive', 'Pictures', 'Saved Games', 'Searches', and 'Videos'. On the right side of the file browser, there is a 'New' button. A dropdown menu is open from this button, showing options: 'Python 3 (ipykernel)', 'Terminal', 'Console', 'New File', and 'New Folder'. A red arrow points to the 'New Folder' option. Below these options, a list of recent files and folders is shown, including 'yesterday', 'last month', and several entries with timestamps like '18 hours ago' and '5 months ago'.



RedHat Home

localhost:8888/tree/Users/dr.eet/ML%20Test

DrEET Copilot Gemini Google Cloud Skills... Artificial Intelligence Moodle RH124 - pr01s02 Hybrid Cloud Devel... Cmp Sc 4760 -- Op...

jupyter

File View Settings Help

Files Running

Rename Delete

/ Users / dr.eet / ML Test /

Name

RedHat.ipynb

test1.ipynb

New Upload

Python 3 (ipykernel)

Terminal

Console

New File

New Folder

Create a new Python file from Python 3

You can easily write and run Python code in Jupyter Notebook.

A screenshot of a Jupyter Notebook interface in a web browser. The browser tabs show 'AD141 - ch02', 'Home', and 'RedHat'. The address bar shows 'localhost:8888/notebooks/ML%20Test/RedHat.ipynb'. The Jupyter logo and 'RedHat Last Checkpoint: 6 hours ago' are in the top left. The top menu bar includes 'File', 'Edit', 'View', 'Run', 'Kernel', 'Settings', and 'Help'. The top toolbar includes icons for saving, opening, and running code. The main area shows a code cell with the following Python code:

```
[3]: num1=int(input("Enter num1"))
      num2=int(input("Enter num1"))
      for num in range(num1,num2):
          print(num)
```

A red arrow points from the 'Run' button in the top toolbar to the code cell. To the right of the code cell, the text 'Write code here.' is displayed with a red arrow pointing to the code. Below the code cell, the output is shown:

```
Enter num1 10
Enter num1 15
10
11
12
13
14
```

A red arrow points from the output to the text 'Running Results'.

Installing scikit-learn



```
pip install numpy scipy matplotlib ipython scikit-learn pandas
```

A screenshot of a Windows Command Prompt window. The title bar reads 'Command Prompt'. The command prompt shows the command 'C:\Users\dr.eet>pip install numpy scipy matplotlib ipython scikit-learn pandas' being entered, with a cursor at the end of the line.

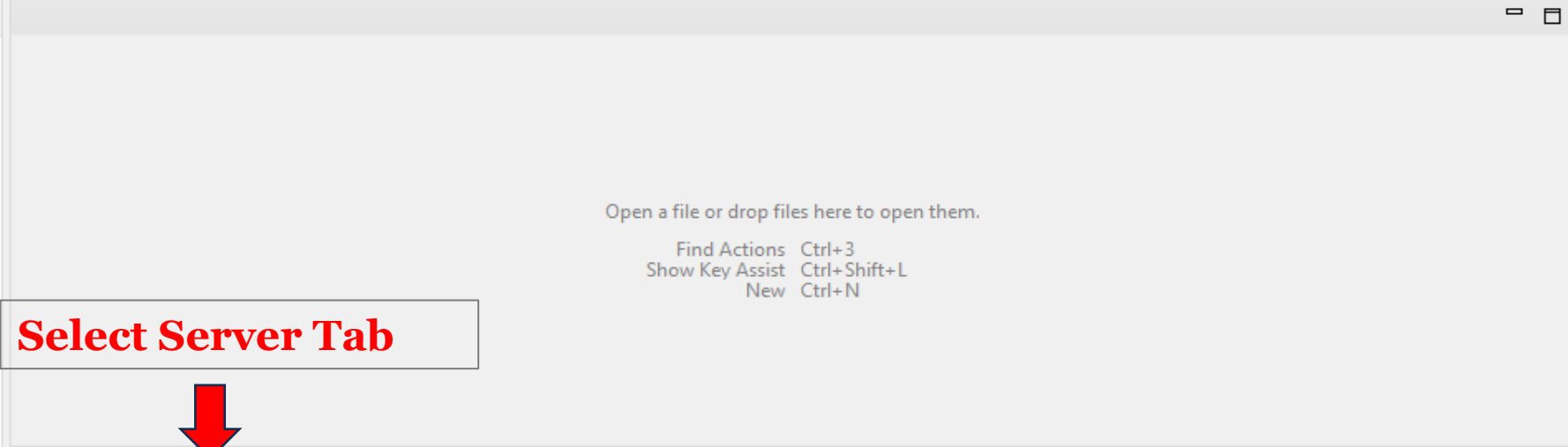
```
C:\Users\dr.eet>pip install numpy scipy matplotlib ipython scikit-learn pandas_
```

I. Configure Tomcat Web Server

Configure Tomcat Web Server

J2EE_2024 - Eclipse IDE

File Edit Navigate Search Project Run Window Help



Outline ×

There is no active editor that provides an outline.

Select Server Tab



Problems Servers × Terminal Data Source Explorer Properties Console

[No servers are available. Click this link to create a new server...](#)

Configure Tomcat Web Server



J2EE_2024 - Eclipse IDE

File Edit Navigate Search Project Run Window Help

Project Explorer

- Project Ex...
- Servers
- TestDB

Problems Servers Terminal

No servers are available. [Click this link to create](#)

New Server

Define a New Server

Choose the type of server to create

Select the server type:

type filter text

- Apache
 - Tomcat v3.2 Server
 - Tomcat v4.0 Server
 - Tomcat v4.1 Server
 - Tomcat v5.0 Server
 - Tomcat v5.5 Server
 - Tomcat v6.0 Server
 - Tomcat v7.0 Server
 - Tomcat v8.0 Server
 - Tomcat v8.5 Server
 - Tomcat v9.0 Server
 - Tomcat v10.0 Server
 - Tomcat v10.1 Server**
- Basic
- IBM

Publishes and runs J2EE, Java EE, and Jakarta EE Web projects and server configurations to a local Tomcat server.

Server's host name: localhost

Server name: Tomcat v10.1 Server at localhost

Server runtime environment: Create a new runtime environment [Add...](#)

[Configure runtime environments...](#)

Choose Tomcat v10.1 Server

Click on Next Button

< Back **Next >** Finish Cancel

Browse Button

Configure Tomcat Web Server

J2EE_2024 - Eclipse IDE

File Edit Navigate Search Project Run Window Help

Project Ex... X

Servers
TestDB

Problems Servers X
No servers are available. Click this link to learn more.

Problems Servers X
No servers are available. Click this link to learn more.

0 items selected

New Server

Select Folder

< > << >> << 2023-2024_UCSTGI > 2205_J2EE > Search 2205_J2EE

Organize New folder

	Name	Date modified	Type
Downloads	eclipse	9/8/2023 3:55 AM	File folder
Documents	Java EE Installer	11/27/2023 9:23 PM	File folder
Pictures	Lecture	11/30/2023 10:20 PM	File folder
Lecture	REF	11/27/2023 12:17 AM	File folder
Prolog Program	Tomcat 10.1	11/22/2023 11:25 AM	File folder
REF			
OneDrive			
OneDrive			
This PC			
Network			

Locate & Select Tomcat 10.1 folder

Folder: Tomcat 10.1

Select Folder

Cancel



< Back

Next >

Finish

Cancel

Outline X

There is no active editor that provides an outline.

Icons for running, debugging, and other IDE functions.

Bottom status bar icons.

Configure Tomcat Web Server

J2EE_2024 - Eclipse IDE

File Edit Navigate Search Project Run Window Help

Project Ex...

Servers
TestDB

Problems Servers

No servers are available. Click this link to download the Tomcat binaries.

0 items selected

New Server

Select Folder

< > << >> 2205_J2EE > Tomcat 10.1 >

Search Tomcat 10.1

Organize New folder

Downloads

Documents

Pictures

Lecture

Lecture

Prolog Program

REF

OneDrive

OneDrive

This PC

Network

Name

Date modified

Type

apache-tomcat-10.1.16

11/11/2023 6:47 AM

File folder

**Select the apache-tomcat-10.1.16 folder
under Tomcat 10.1 folder**

Folder: apache-tomcat-10.1.16

Select Folder

Cancel



< Back

Next >

Finish

Cancel

Outline

There is no active editor that provides an outline.

Icons for various IDE features like Run, Debug, and Search.

Bottom status bar with icons for Help, Run, and other IDE functions.

Configure Tomcat Web Server

J2EE_2024 - Eclipse IDE

File Edit Navigate Search Project Run Window Help

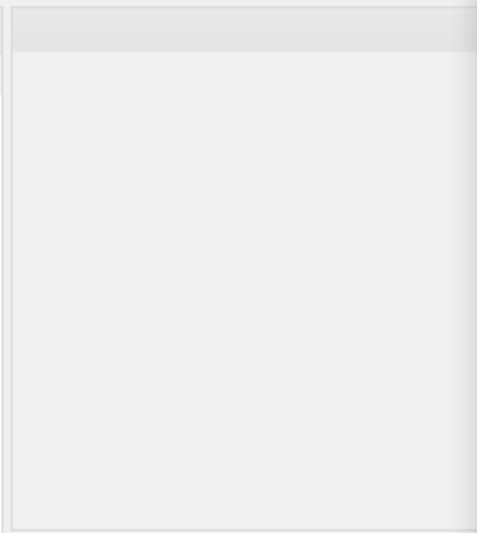


Project Ex... X



> Servers

> TestDB



Problems Servers X Terminal

No servers are available. Click this link to create

0 items selected

New Server

Tomcat Server

Specify the Tomcat installation directory and JRE for this runtime. The specified JRE controls the highest supported Java Facet version.

Name:

Apache Tomcat v10.1 (2)

Tomcat installation directory:

D:\1_Teaching\2023-2024_UCSTGI\2205_J2EE\Tomcat 10.1\apache

Browse...

apache-tomcat-10.1.12

Download and Install...

JRE:

Workbench default JRE

Installed JREs...

Click on Finish Button

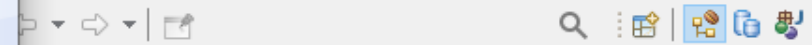
?

< Back

Next >

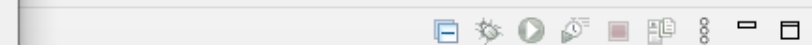
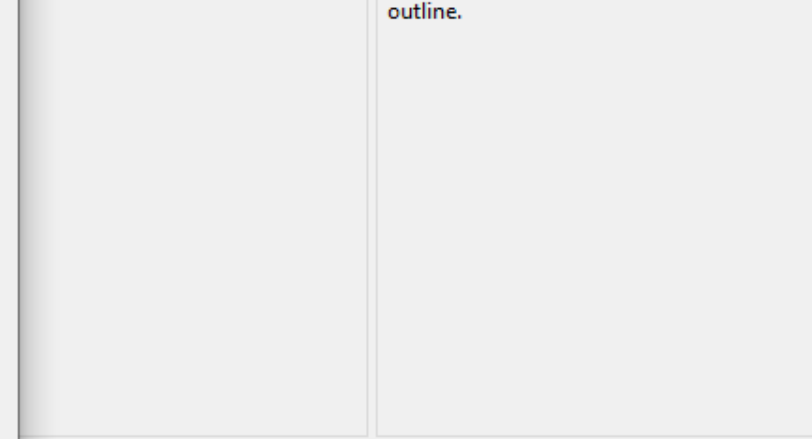
Finish

Cancel



Outline X

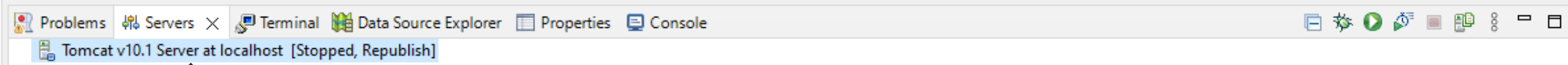
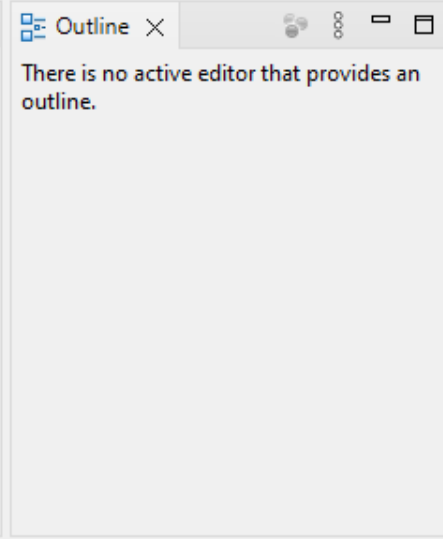
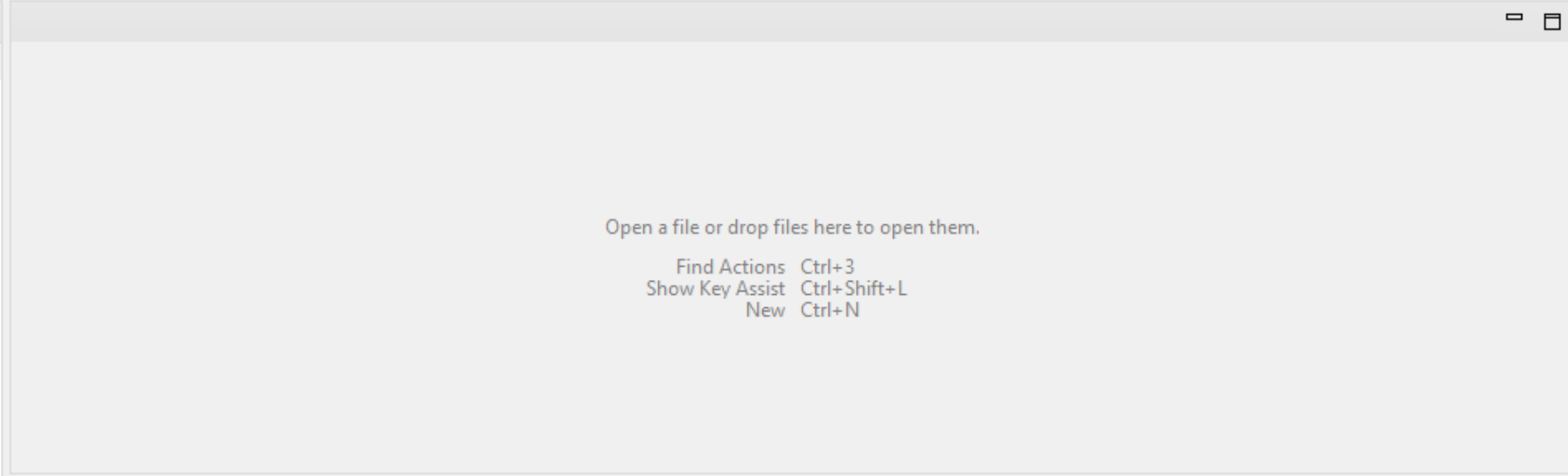
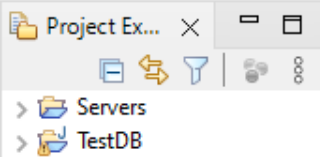
There is no active editor that provides an outline.



Configure Tomcat Web Server

J2EE_2024 - Eclipse IDE

File Edit Navigate Search Project Run Window Help



**Tomcat Server under
Server Tab**

Configure Tomcat Web Server

J2EE_2024 - Eclipse IDE

File Edit Navigate Search Project Run Window Help

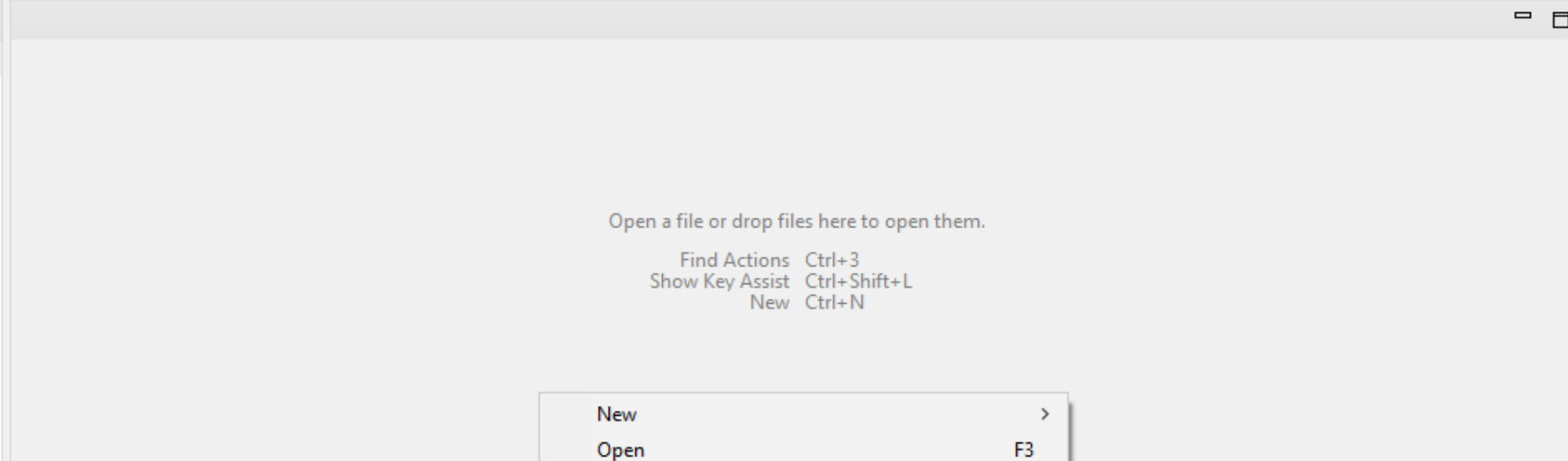


Project Ex... X



> Servers

> TestDB

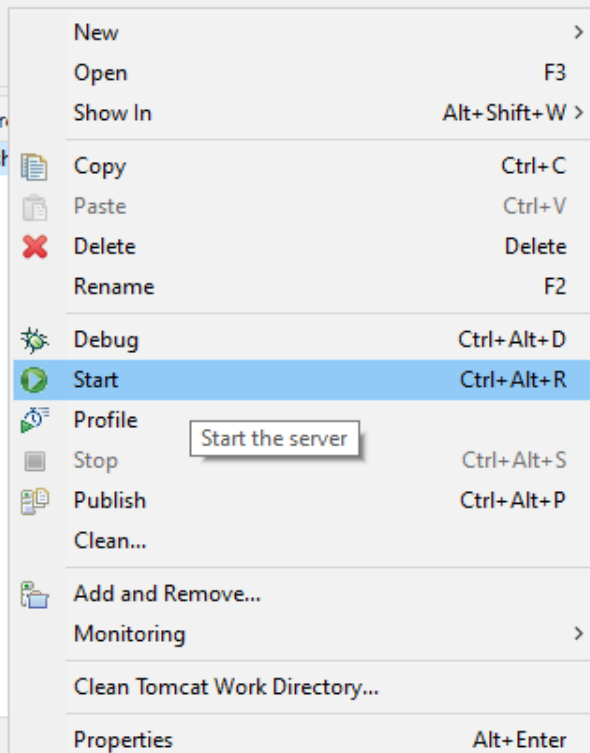


Outline X

There is no active editor that provides an outline.

Problems Servers X Terminal Data Source

Tomcat v10.1 Server at localhost [Stopped, Republish]



**Right Click on Tomcat
Server and Choose Start
from Menu**

1 item selected

Configure Tomcat Web Server

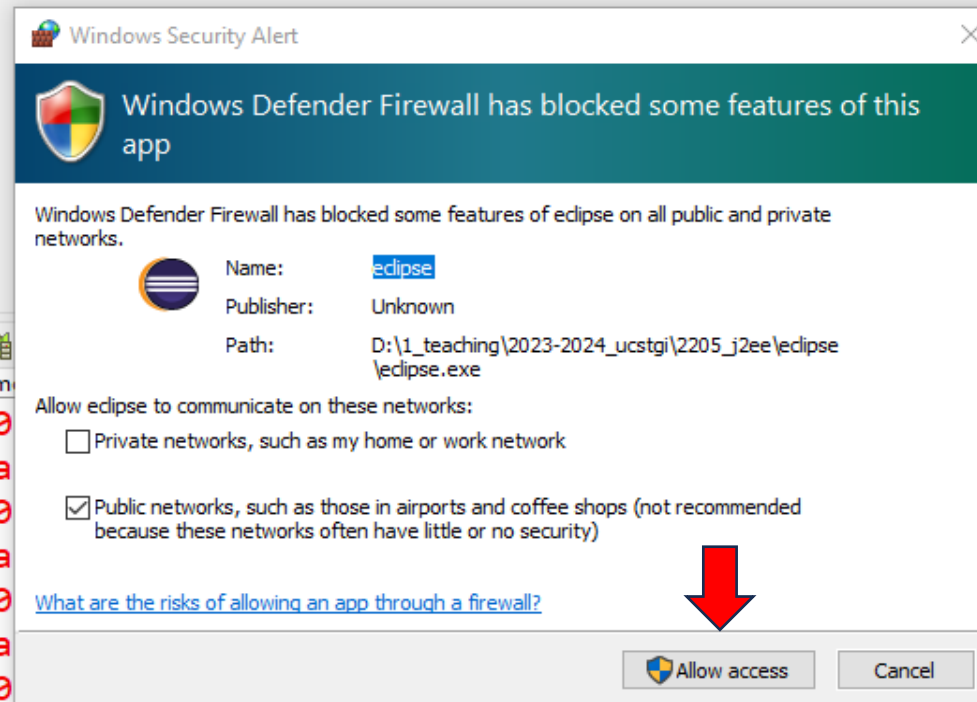
J2EE_2024 - Eclipse IDE

File Edit Navigate Search Project Run Window Help



Project Ex...
Servers
TestDB

Outline X
There is no active editor that provides an outline.



Problems Servers Terminal

Tomcat v10.1 Server at localhost [Apache Tomcat v10.1.0]

```
Nov 30, 2023 10:32:00 PM org.apache.catalina.core.AprLifecycleListener lifecycleEvent
INFO: Command line argument: -XX:+ShowCodeDetailsInExceptionMessages
Nov 30, 2023 10:32:00 PM org.apache.catalina.core.AprLifecycleListener lifecycleEvent
INFO: Command line argument: -XX:+ShowCodeDetailsInExceptionMessages
Nov 30, 2023 10:32:00 PM org.apache.catalina.core.AprLifecycleListener lifecycleEvent
INFO: Command line argument: -XX:+ShowCodeDetailsInExceptionMessages
Nov 30, 2023 10:32:00 PM org.apache.catalina.core.AprLifecycleListener lifecycleEvent
INFO: Command line argument: -XX:+ShowCodeDetailsInExceptionMessages
Nov 30, 2023 10:32:07 PM org.apache.catalina.core.AprLifecycleListener lifecycleEvent
INFO: Loaded Apache Tomcat Native library [2.0.6] using APR version [1.7.4].
```

Starting Tomcat v10.1 S...calhost: (100%)

Configure Tomcat Web Server

J2EE_2024 - Eclipse IDE

File Edit Navigate Search Project Run Window Help

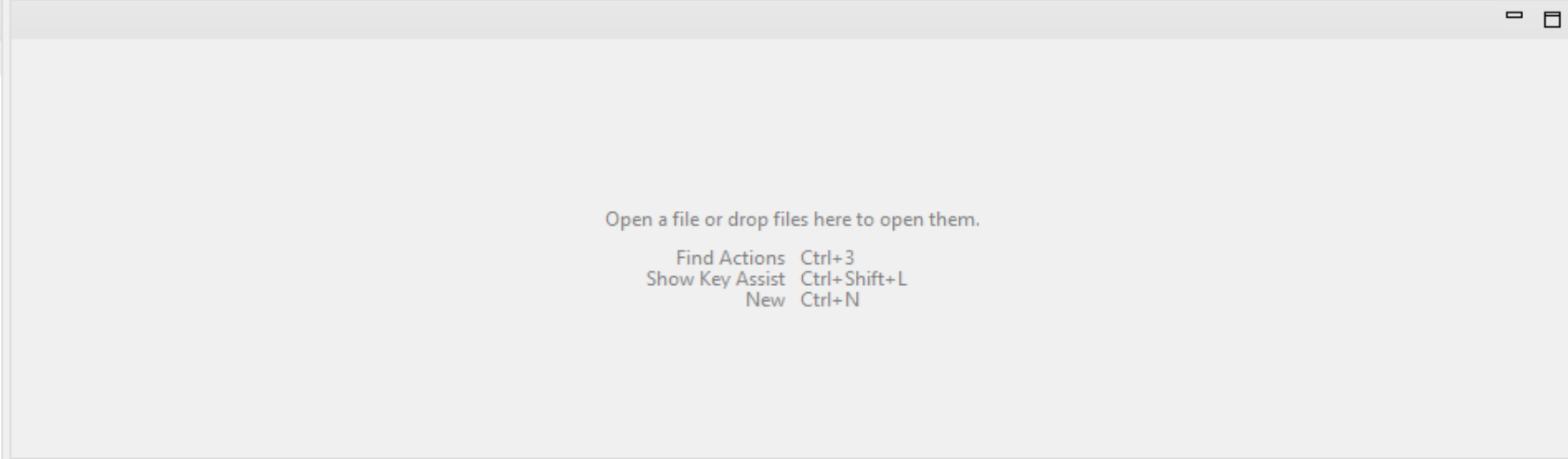


Project Ex... X



> Servers

> TestDB



Outline X

There is no active editor that provides an outline.

Problems Servers X Terminal Data Source Explorer Properties Console

Tomcat v10.1 Server at localhost [Started, Synchronized]



**Server is started and
successfully installed**

1 item selected



II. Creating a Dynamic Web Application

Creating a Web Application

J2EE_2024 - Eclipse IDE

File Edit Navigate Search Project Run Window Help

New **Alt+Shift+N**

Open File...

Open Projects from File System...

Recent Files

Close Editor **Ctrl+W**

Close All Editors **Ctrl+Shift+W**

Save **Ctrl+S**

Save As...

Save All **Ctrl+Shift+S**

Revert

Move...

Rename... **F2**

Refresh **F5**

Convert Line Delimiters To

Print... **Ctrl+P**

Import...

Export...

Properties **Alt+Enter**

Switch Workspace

Restart

Exit

Maven Project

Enterprise Application Project

Dynamic Web Project

EJB Project

Connector Project

Application Client Project

Static Web Project

JPA Project

Project...

CSS File

JavaScript File

Servlet

Session Bean (EJB 3.x/4.x)

Message-Driven Bean (EJB 3.x/4.x)

Web Service

HTML File

XML File

Folder

File

JSP File

Example...

Other... **Ctrl+N**

Create a Dynamic Web project



**File
New
Dynamic Web Project**



Outline

There is no active editor that provides an outline.

Console

TestDB

Creating a Web Application



JAKARTA EE

J2EE_2024 - Eclipse IDE

File Edit Navigate Search Project Run Window Help

Project Ex... X

Servers

TestDB

Problems Servers X

Tomcat v10.1 Server at localh

New Dynamic Web Project

Dynamic Web Project

Create a standalone Java-based Web Application or add it to a new or existing Enterprise Application.



Project name: Test

Project location

☒ Use default location

Location: D:\J2EE_2024\Test

Browse...

Target runtime

Apache Tomcat v10.1 (2)

New Runtime...

Dynamic web module version

6.0

Configuration

Default Configuration for Apache Tomcat v10.1 (2)

Modify...

A good starting point for working with Apache Tomcat v10.1 (2) runtime. Additional facets can later be installed to add new functionality to the project.

EAR membership

☐ Add project to an EAR

EAR project name: TestEAR

New Project...

Working sets

☐ Add project to working sets

New...

Working sets:

Select...



< Back

Next >

Finish

Cancel

Outline X

There is no active editor that provides an outline.

Icons for various IDE features

Creating a Web Application



J2EE_2024 - Eclipse IDE

File Edit Navigate Search Project Run Window Help



Project Ex... X



> Servers

> TestDB

Problems Servers X

Tomcat v10.1 Server at localh

New Dynamic Web Project

Java

Configure project for building a Java application.

Source folders on build path:

src\main\java

Add Folder...

Edit...

Remove

Default output folder:

build\classes



< Back

Next >

Finish

Cancel

Outline X

There is no active editor that provides an outline.

Creating a Web Application

J2EE_2024 - Eclipse IDE

File Edit Navigate Search Project Run Window Help

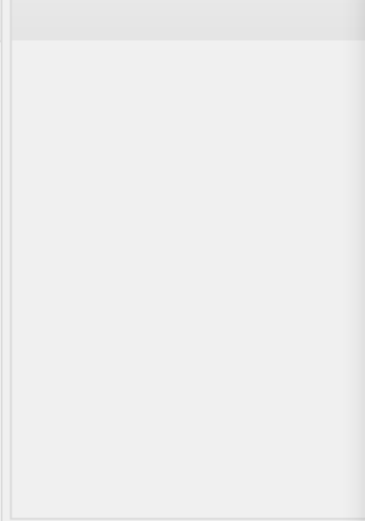


Project Ex... X



Servers

TestDB



Problems Servers X

Tomcat v10.1 Server at localh

TestDB

New Dynamic Web Project

Web Module

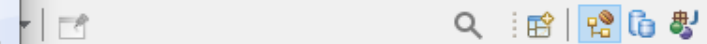
Configure web module settings.

Context root:

Content directory:

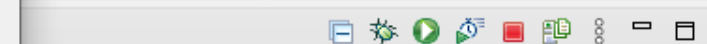
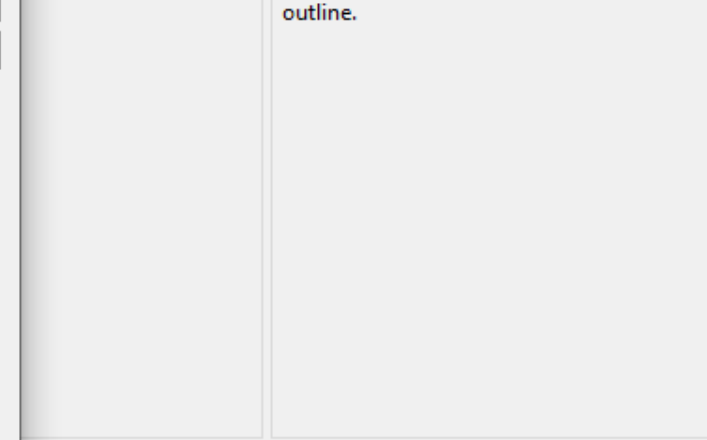
☒ Generate web.xml deployment descriptor

check on Check Box



Outline X

There is no active editor that provides an outline.



REFERENCES

<https://www.pragimtech.com/blog/blazor/what-are-restful-apis/>

<https://realpython.com/api-integration-in-python/>